Reddit Unified Media Metadata Store Architecture

100k RPS at 17ms p99 on AWS Aurora PostgreSQL Without Read Caching

104,000 RPS	16.8 ms
PEAK PRODUCTION METADATA REQUEST RATE	VERIFIED P99 END-TO-END LATENCY CEILING

1. The Problem: Disparate Media Metadata Silos

Reddit serves tens of billions of media impressions daily across feed rendering, video streaming, and image carousels. Historically, media attributes (bitrate ladders, HLS playlists, aspect ratios, thumbnails) were fragmented across disparate legacy databases: Cassandra for video transcoding states, DynamoDB for image dimensions, and legacy Postgres tables. This fragmentation prevented cross-media relational integrity and caused mobile client rendering inconsistencies.

2. The Architecture Decision: Eliminating the Read-Through Cache

Conventional system design suggests placing a multi-node Redis cluster in front of the database to absorb 100k RPS. Reddit explicitly rejected an external caching layer:

- **Cache Invalidation Bugs:** Media attributes mutate continuously as async transcoding jobs finish and moderation actions trigger. Cache-database drift caused severe UI anomalies.
- **Thundering Herd Contention:** Viral posts on r/all caused massive cache stampedes when TTL expired, threatening backend database stability.
- **Operational Overhead:** Eliminating Redis saved millions in RAM infrastructure while guaranteeing strict ACID read-your-writes consistency directly from the primary datastore.

3. AWS Aurora PostgreSQL Storage Architecture

To serve 100k RPS directly from PostgreSQL, Reddit leveraged AWS Aurora's decoupled storage engine:

```
Log is the Database: Compute Nodes write only Redo Log records to a 6-way replicated Quorum Storage Fleet.
```

Aurora read replicas do not perform write-amplifying page flushes; they stream log records continuously from shared distributed storage, maintaining hot in-memory buffer caches with sub-10ms replica lag.

4. PgBouncer Architecture & Transaction Pooling

Standard PostgreSQL allocates a dedicated OS process per client connection. With thousands of application pods, connection memory overhead and CPU context switching degrade database throughput.

Transaction Pooling Multiplexing Formula

`30,000+ Application Worker Connections → [PgBouncer Proxy (Transaction Mode)] → 120 Backend Server Connections`

In transaction pooling mode, a server database connection is bound only for the active lifespan of a single SQL query. Idle worker connections consume zero database CPU or RAM, eliminating connection exhaustion.

5. Relational Schema & Index-Only Scans

Reddit designed a unified table partitioned by creation epoch, complemented by specialized covering B-Tree indexes:

```
-- High-performance covering index definition
CREATE INDEX idx_media_post_status ON media_assets (post_id, status)
INCLUDE (variants, width, height, duration_ms);
```

The `INCLUDE` clause stores payload metadata directly in the index leaf pages. PostgreSQL satisfies lookups via an **Index-Only Scan**, bypassing secondary heap table lookups entirely and returning query results in **< 1.5 milliseconds per row**.

6. Kafka CDC Zero-Downtime Migration Pipeline

Billions of records were migrated from Cassandra and DynamoDB via dual-write Kafka event streaming. A background shadow validator compared primary read traffic between Cassandra and Aurora for six months, verifying 100% data fidelity before the final DNS cutover.

7. Empirical Production Latency Telemetry

Operating State	Request Throughput	p50 Latency	p95 Latency	p99 Latency
Baseline Normal Traffic	42,000 RPS	3.8 ms	8.2 ms	12.4 ms
Viral Front-Page Surge	**104,000 RPS**	**4.9 ms**	**11.5 ms**	**16.8 ms**

8. Architectural Comparison: Cache vs. Tuned Relational

Design Vector	Redis Cache + Postgres DB	Direct Aurora PostgreSQL (Reddit Model)
Data Consistency	Eventual / Subject to invalidation lag.	**Strict ACID Read-Your-Writes.**
Thundering Herd Risk	High (Requires distributed locks).	**Zero (No cache layer to miss).**
Infrastructure Footprint	Two distinct distributed systems.	**Single unified relational cluster.**
Operational Complexity	Cache warmup, TTL tuning, sync jobs.	**PostgreSQL schema & index management.**

9. Production Hardening Lessons

1. Tune Autovacuum for High Update Rates
PostgreSQL MVCC generates dead tuples on status updates. Reddit configured aggressive autovacuum scale factors (`autovacuum_vacuum_scale_factor = 0.02`) to prevent table bloat from slowing down index scans.

2. Guard against Pooler Statement Timeouts
Configure strict 50ms statement timeouts in PgBouncer. If a single unindexed query slips through, the pooler terminates it immediately, preventing queue exhaustion across the entire cluster.